Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 46: *Dimensionality Reduction, Principled Component Analysis (PCA)*
November 10, 2026

CC BY Licence 4.0: Lau Møller Andersen 2026

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects?

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous?

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data?

Week 47:

Lesson 9: Neural networks

Class 9: Code review

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything!

CC BY Licence 4.0: Lau Møller Andersen 2026

Recap

- The space of hyperparameters can be quite large
  - Therefore we need consistent ways of exploring them
- We can use *scikit-learn* to create high-level structures to run classification analyses
- We had a look at optimising PCA
  - but we didn’t thoroughly look at what it is

CC BY Licence 4.0: Lau Møller Andersen 2026

Learning goals and outline

Dimensionality Reduction, Principled Component Analysis (PCA)

1) Learning how we can extract the features that explain the most variance
2) Understanding how that can improve classification
3) Get acquainted with the concept of a eigenvector

CC BY Licence 4.0: Lau Møller Andersen 2026

The curse of dimensionality

```
import numpy as np
import matplotlib.pyplot as plt
from os.path import join

path = '/home/lau/Skrivebord/class_subject'
data = np.load(join(path, 'megmag_data.npy'))

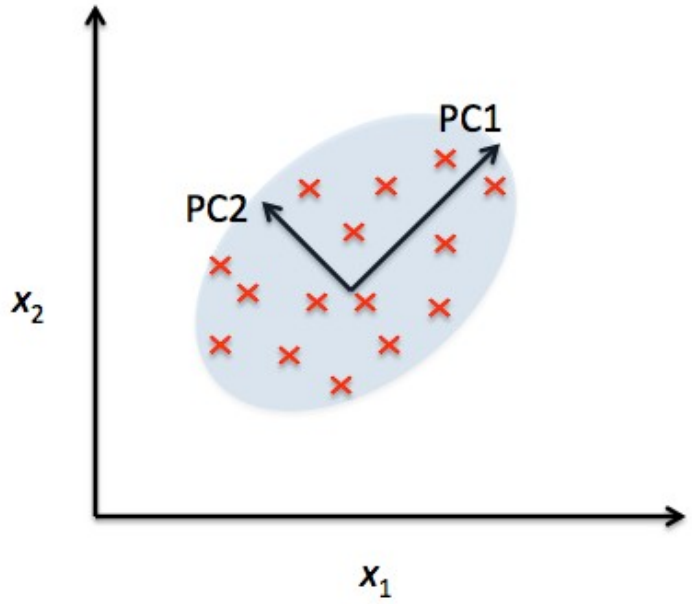
print('Shape: ' + str(data.shape))
print('n measurements: ' + str(np.prod(data.shape)))
print('n observations: ' + str(data.shape[0]))
print('n features: ' + str(np.prod(data.shape[1:])))
```

```
Shape: (682, 102, 251)
n measurements: 17460564
n observations: 682
n features: 25602
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Principled components

FINDING THE DIRECTIONS OF MOST VARIANCE

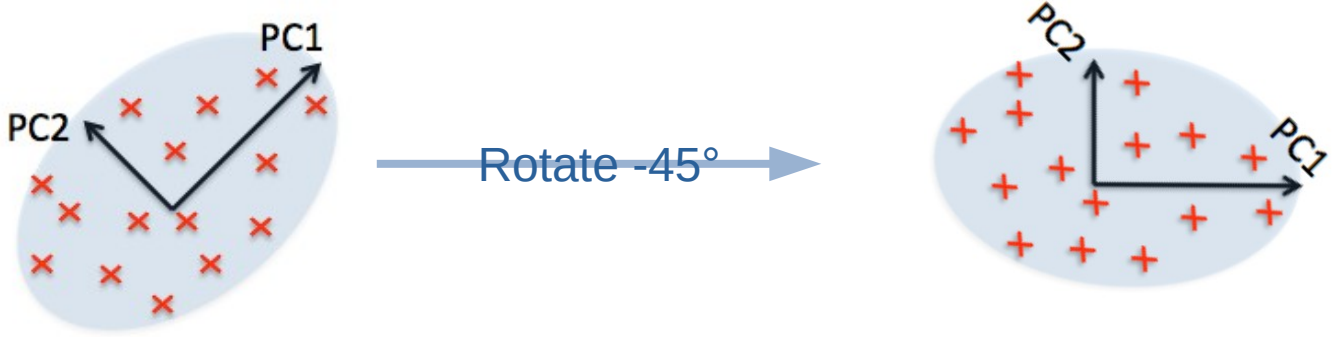


(p. 128: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Principled components

FINDING THE DIRECTIONS OF MOST VARIANCE



PC1 has the most variance

(p. 128: Raschka, 2015)

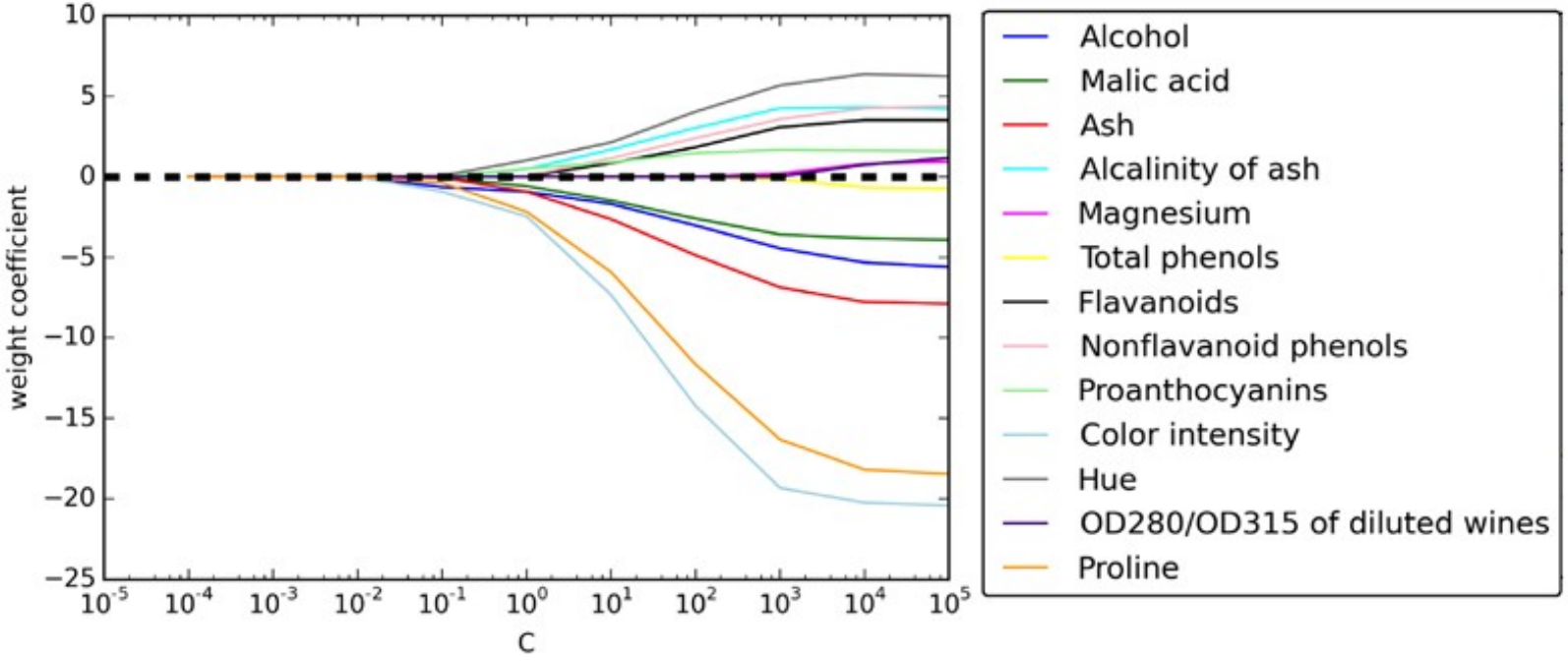
CC BY Licence 4.0: Lau Møller Andersen 2026

This can be **generalized** to as many dimensions as you like

CC BY Licence 4.0: Lau Møller Andersen 2026

A dataset (wine)

DIFFERENT FROM THE ONE USED LAST TIME



(p. 118: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

```
## import wine data
import pandas as pd
url = 'https://archive.ics.uci.edu/ml/' + \
        'machine-learning-databases/wine/wine.data'
df_wine = pd.read_csv(url, header=None)
```

```
X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values
print(X.shape)
print(np.unique(y))
```

```
(178, 13)
[1 2 3]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

AIM: for $\boldsymbol{x}$: find $\boldsymbol{W}$ ($k \leq d$) such that :

$$\boldsymbol{x} = [x_1, x_2, \ldots, x_d], \boldsymbol{x} \in \mathbb{R}^{\boldsymbol{d}}$$

$$\downarrow \boldsymbol{xW}, \boldsymbol{W} \in \mathbb{R}^{d \times k}$$

$$\boldsymbol{z} = [z_1, z_2, \ldots, z_k], \boldsymbol{z} \in \mathbb{R}^{\boldsymbol{k}}$$

CC BY Licence 4.0: Lau Møller Andersen 2026

Approach

PRINCIPLED COMPONENT ANALYSIS

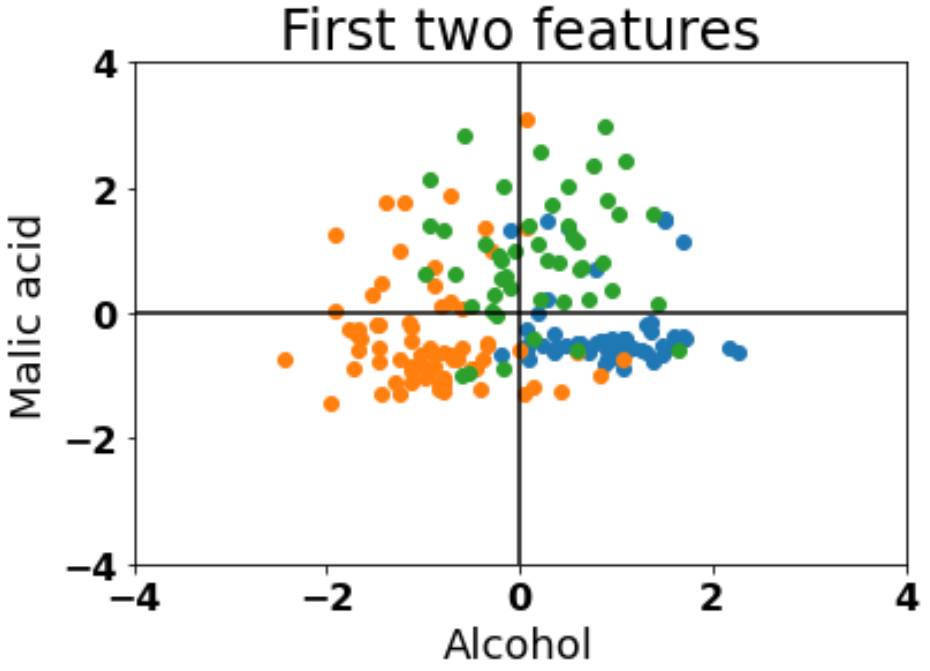
1) Standardize the *d*-dimensional dataset
2) Construct the covariance matrix
3) Decompose the covariance matrix into its eigenvectors and eigenvalues
4) Select *k* eigenvectors that correspond to the *k* largest eigenvalues where *k* is the dimensionality of the new feature subspace ($k \leq d$)
5) Construct a projection matrix ***W*** from the “top” *k* eigenvectors
6) Transform the *d*-dimensional input dataset ***X*** using the projection matrix ***W*** to obtain the new *k*-dimensional feature subspace

(p. 129: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Standardize the dataset (1)

FINDING THE DIRECTIONS OF MOST VARIANCE



CC BY Licence 4.0: Lau Møller Andersen 2026

Construct the covariance matrix (2)

FINDING THE DIRECTIONS OF MOST VARIANCE

$$\sigma_{jk} = \frac{1}{n} \sum_{i=1}^{n} \left(x_j^{(i)} - \mu_j \right) \left(x_k^{(i)} - \mu_k \right)$$

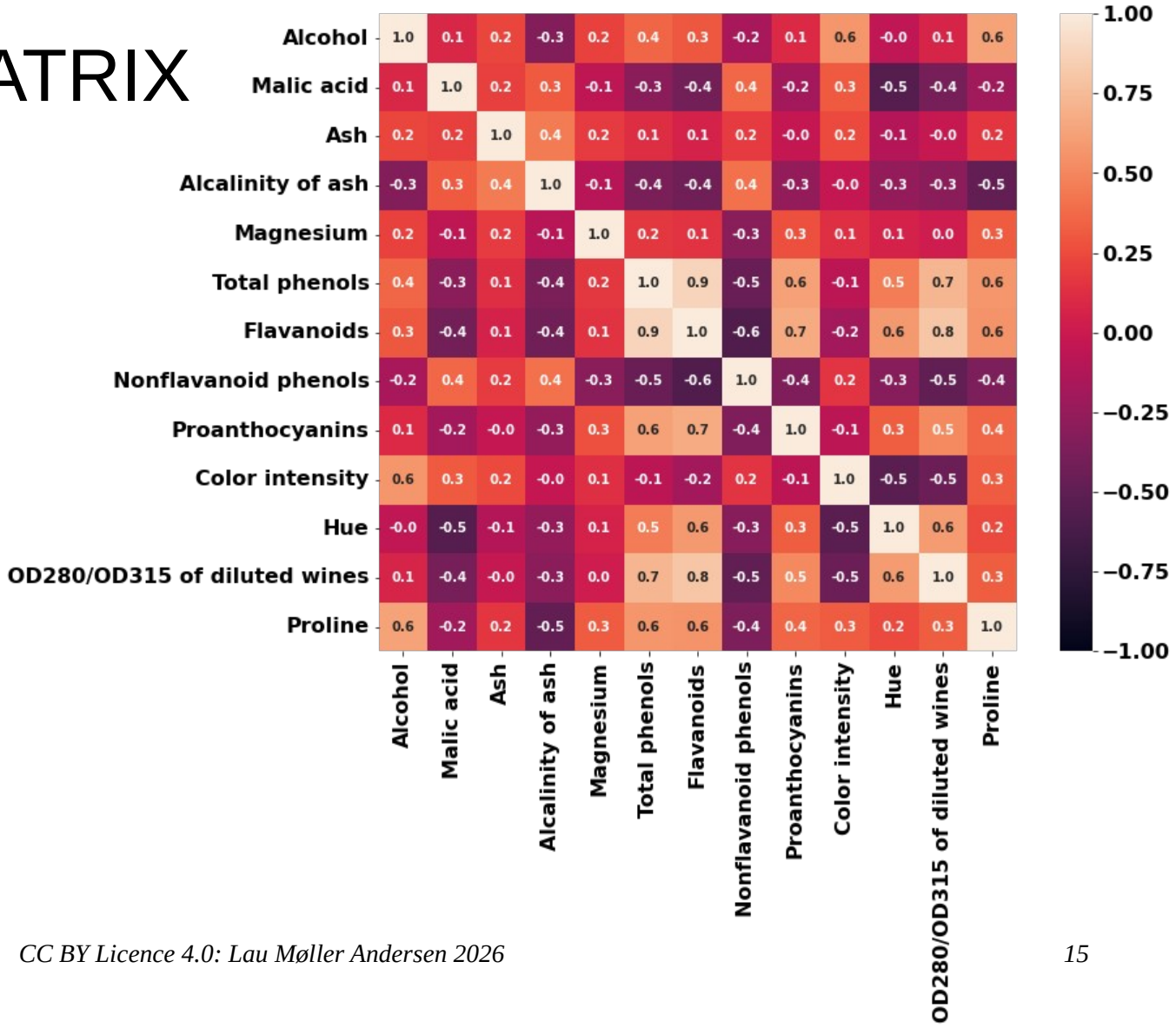
$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 \end{bmatrix}$$

Questions:
What will be the size of the covariance matrix in the *Wine* dataset?

What is μ equal to in a standardised dataset?

CC BY Licence 4.0: Lau Møller Andersen 2026

COVARIANCE MATRIX



CC BY Licence 4.0: Lau Møller Andersen 2026

Gentle introduction to PCA

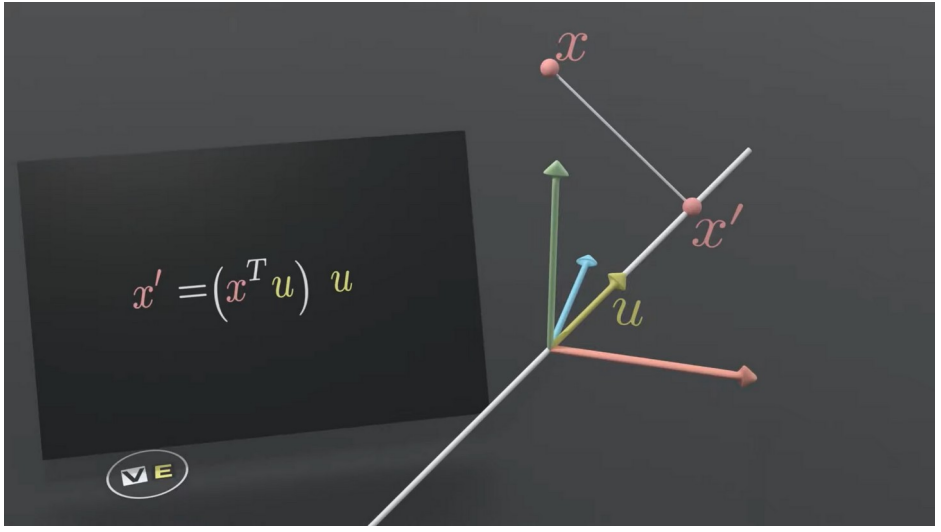
/home/lau/Videoklip/methods3/Principal Component Analysis (PCA) [FD4DeN81ODY].mp4
Principal Component Analysis by Visually Explained (Youtube)

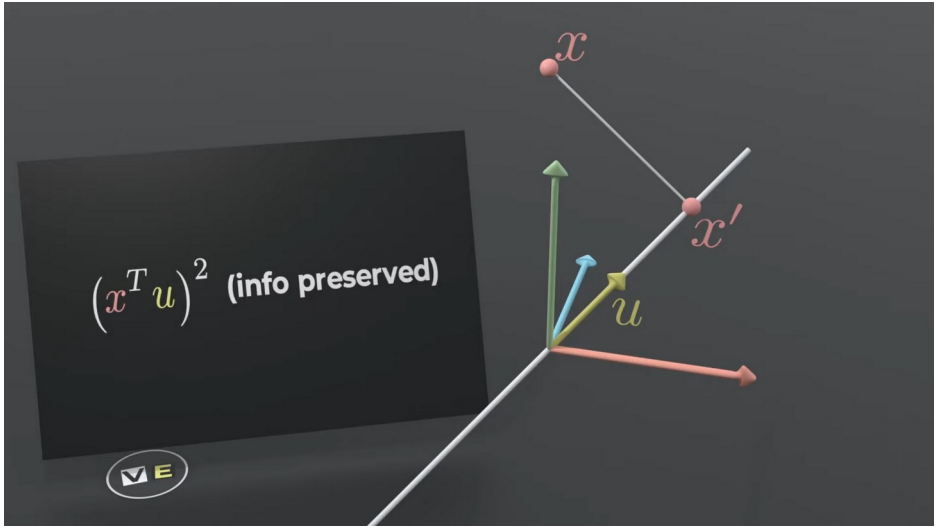
- Questions:

1) What does it mean to preserve as much information as possible?
2) Why is it wasteful to project all points onto one axis (e.g. GDP)
3) What is a projection? $x_i' = (x_i^T u)u$
    1) What do you get from solving this? $\max \sum_{i=1}^{n} (x_i^T u)^2$
4) What is a Lagrange multiplier?
5) What are eigenvectors and -values?

CC BY Licence 4.0: Lau Møller Andersen 2026

Projection





/home/lau/Videoklip/methods3/Principal Component Analysis (PCA) [FD4DeN81ODY].mp4
Principal Component Analysis by Visually Explained (Youtube)

CC BY Licence 4.0: Lau Møller Andersen 2026

Derivation

(1) $\max \sum_{i=1}^{n} (\boldsymbol{x}_i^T \boldsymbol{u})^2$

(2) $\max \sum_{i=1}^{n} \boldsymbol{x}_i^T \boldsymbol{u} \boldsymbol{x}_i^T \boldsymbol{u}$

(3) $\max \sum_{i}^{n} \boldsymbol{u}^T \boldsymbol{x}_i \boldsymbol{x}_i^T \boldsymbol{u}$

(4) $\max \boldsymbol{u}^T \sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^T \boldsymbol{u}$

(5) $\max \boldsymbol{u}^T \boldsymbol{C} \boldsymbol{u}$

$\boldsymbol{u}^T \boldsymbol{u} = 1$: unit vector (length=1)

$\boldsymbol{C} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^T$: covariance matrix

CC BY Licence 4.0: Lau Møller Andersen 2026

Derivation of optimisee

unit vector: $u^T u = 1$

covariance matrix: $C = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T$

maximise variance retained: $\max \sum_{i=1}^{n} (x_i^T u)^2$

write out the square: $\max \sum_{i=1}^{n} x_i^T u x_i^T u$

change the order of the first two factors: $\max \sum_{i=1}^{n} u^T x_i x_i^T u$

move u outside: $\max u^T \sum_{i=1}^{n} x_i x_i^T u$

insert covariance C : $\max u^T C u$

CC BY Licence 4.0: Lau Møller Andersen 2026

Lagrange multipliers

- Useful in optimisation problems with constraints
  - optimisee: $\max \sum_{i=1}^{n} (\boldsymbol{x}_i{}^T \boldsymbol{u})^2$
  - constraint: $\boldsymbol{u}^T \boldsymbol{u} = 1$
- Subtract the constraint equation set equal to 0 (multiplied by a constant, λ) from the optimisee
- Then, find the derivative for u (set it to zero (find the extremum)
- And *lo and behold*, the u that maximises the optimisee is something you might know from Methods 2 – namely the eigenvector

$$L(\boldsymbol{u}) = \boldsymbol{u}^T \boldsymbol{C} \boldsymbol{u} - \lambda(\boldsymbol{u}^T \boldsymbol{u} - 1)$$

$$\nabla L(\boldsymbol{u}) = \nabla(\boldsymbol{u}^T \boldsymbol{C} \boldsymbol{u}) - \lambda \nabla(\boldsymbol{u}^T \boldsymbol{u} - 1) = 0$$

$$2\boldsymbol{C}\boldsymbol{u} - 2\lambda \boldsymbol{u} = 0$$

$$\boldsymbol{C}\boldsymbol{u} = \lambda \boldsymbol{u}$$

CC BY Licence 4.0: Lau Møller Andersen 2026

Find the eigenvectors and eigenvalues (3)

FINDING THE DIRECTIONS OF MOST VARIANCE

$$\Sigma \boldsymbol{v} = \lambda \boldsymbol{v}$$

$\boldsymbol{v}$: eigenvector (a unit vector)
λ: eigenvalue
Σ: covariance matrix

Link to Chris Mathys's lecture from Methods II

Link to 3blue1brown's video of the same matter

CC BY Licence 4.0: Lau Møller Andersen 2026

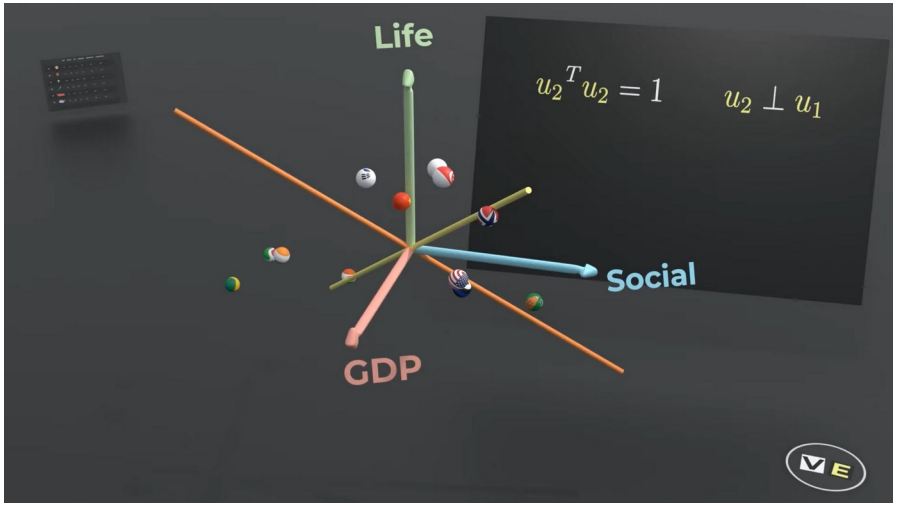
Lagrange

https://www.youtube.com/watch?v=hQ4UNu1P2kw

CC BY Licence 4.0: Lau Møller Andersen 2026

Orthogonality

$$\boldsymbol{u}_n \perp \boldsymbol{u}_{n-1} \ldots \boldsymbol{u}_2 \perp \boldsymbol{u}_1$$



/home/lau/Videoklip/methods3/Principal Component Analysis (PCA) [FD4DeN81ODY].mp4
Principal Component Analysis by Visually Explained (Youtube)

CC BY Licence 4.0: Lau Møller Andersen 2026

After PCA

```
## plotting eigenvalue plots

from sklearn.decomposition import PCA

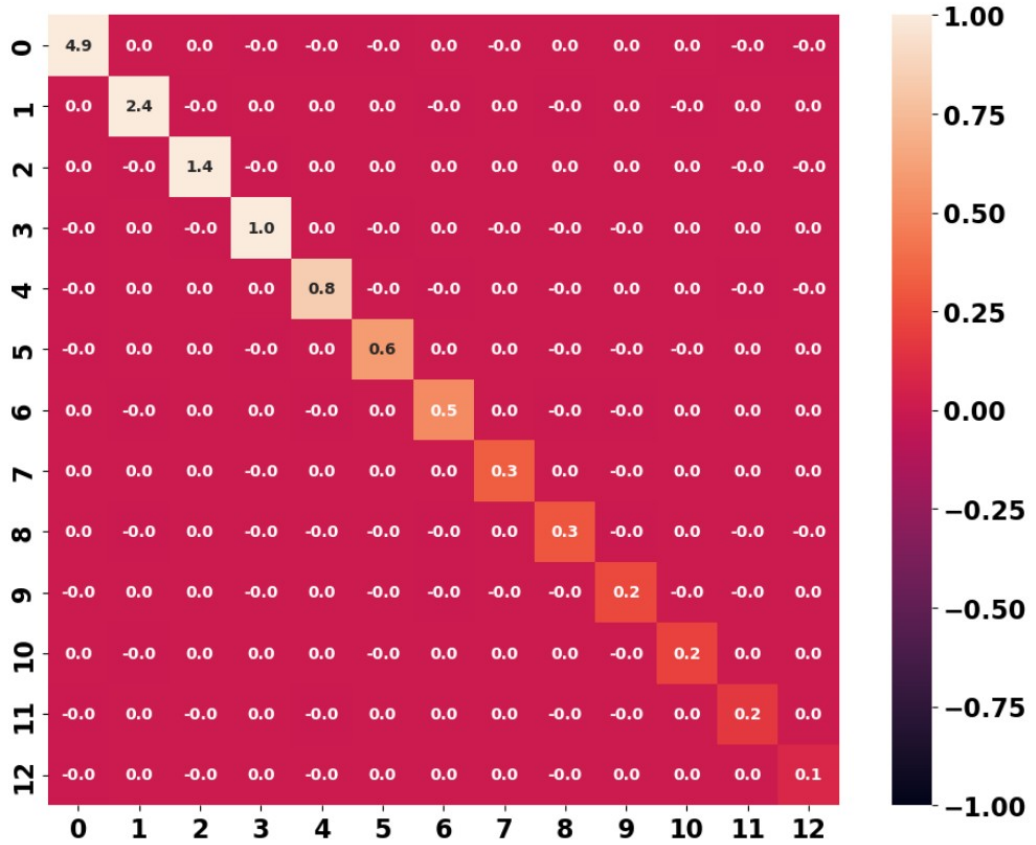
pca = PCA(n_components=X_train.shape[1])

X_train_std_pca = pca.fit_transform(X_train_std)

n = X_train.shape[0]

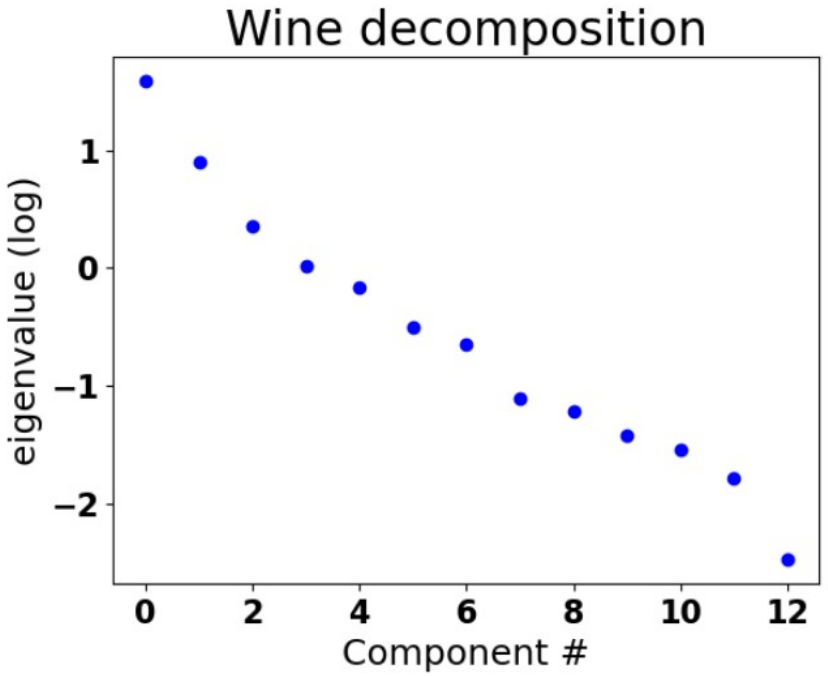
cov_pca = 1/n * X_train_std_pca.T @ X_train_std_pca

plt.figure(figsize=(12, 9))
sns.heatmap(cov_pca, cbar=True, annot=True, square=True,
            fmt='.1f',
            vmin=-1.0, vmax=1.0)
```



CC BY Licence 4.0: Lau Møller Andersen 2026

After PCA

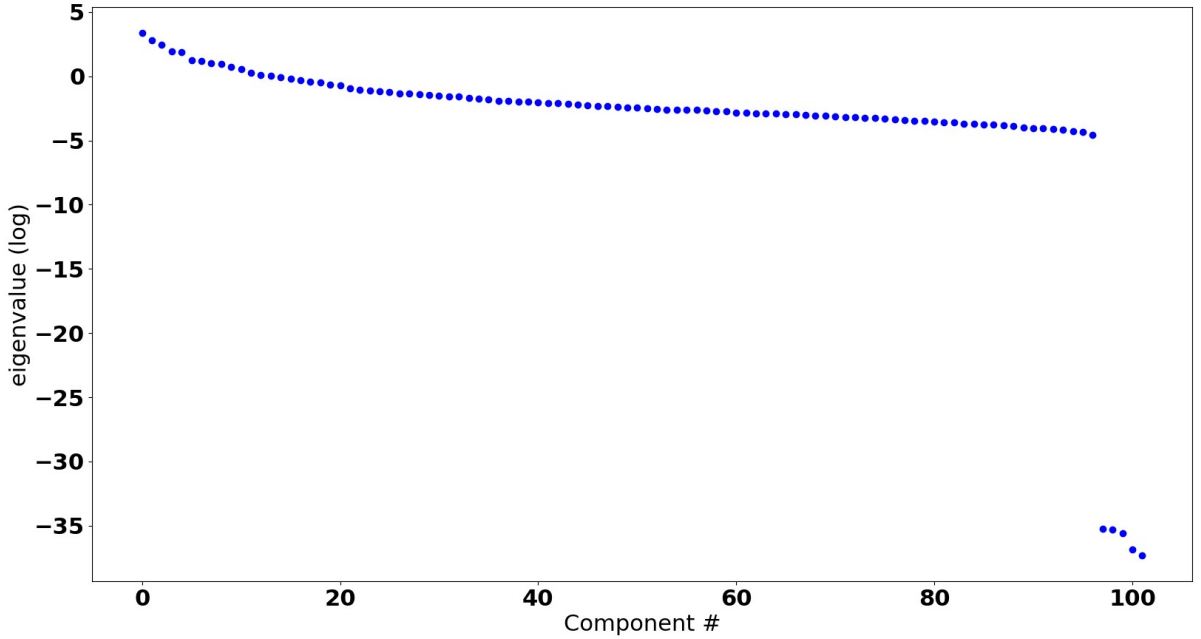



```
##
eigenvalues = pca.explained_variance_
plt.figure()
plt.plot(np.log(eigenvalues), 'bo')
plt.xlabel('Component #')
plt.ylabel('eigenvalue (log)')
plt.title('Wine decomposition')
plt.show()
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Why the jump?
ANOTHER DATASET

Discussion groups:
What might cause a jump like this?



CC BY Licence 4.0: Lau Møller Andersen 2026

Interim summary

- Doing PCA will allow one only to keep the dimensions of the data that contain signal (structured variance) while leaving out the noise dimensions
  - And will allow one to rank the components by variance explained
- Doing this can preventing overfitting of the data at the training stage
  - Which means that it will generalise better to the unseen test data
  - (Overfitting: fitting non-signal components, i.e. noise)

CC BY Licence 4.0: Lau Møller Andersen 2026

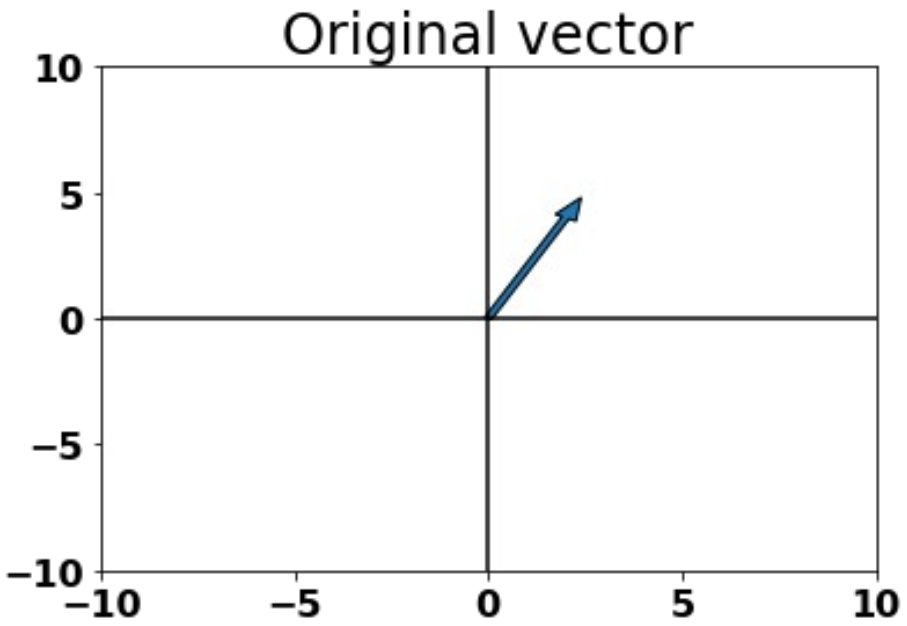
Transformations

Remember: matrix multiplication of a vector can be seen as a **transformation** of the vector

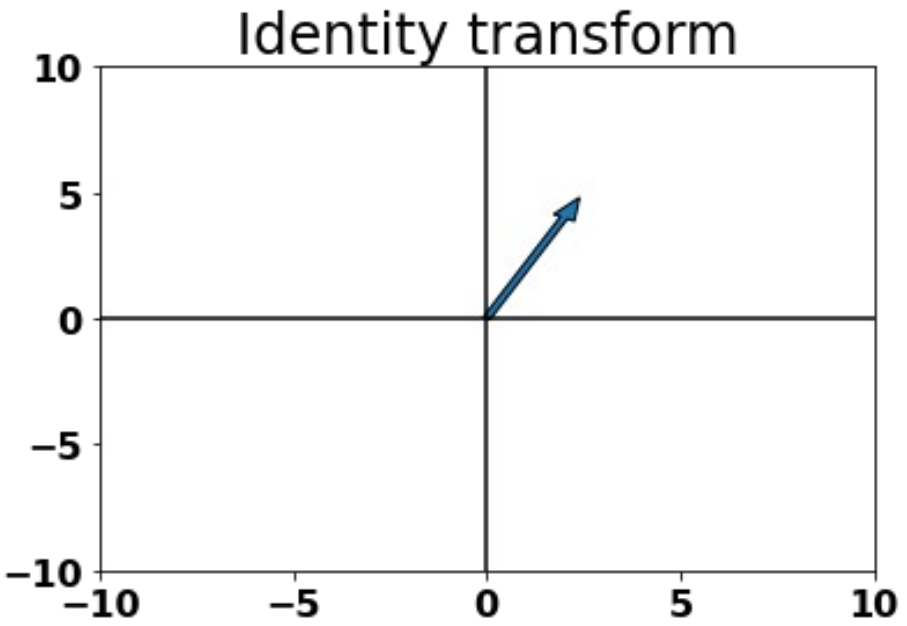
$$T(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$$

$\boldsymbol{A}$: transformation matrix
$\boldsymbol{x}$: column vector
$\boldsymbol{T}$: transformed column vector

CC BY Licence 4.0: Lau Møller Andersen 2026



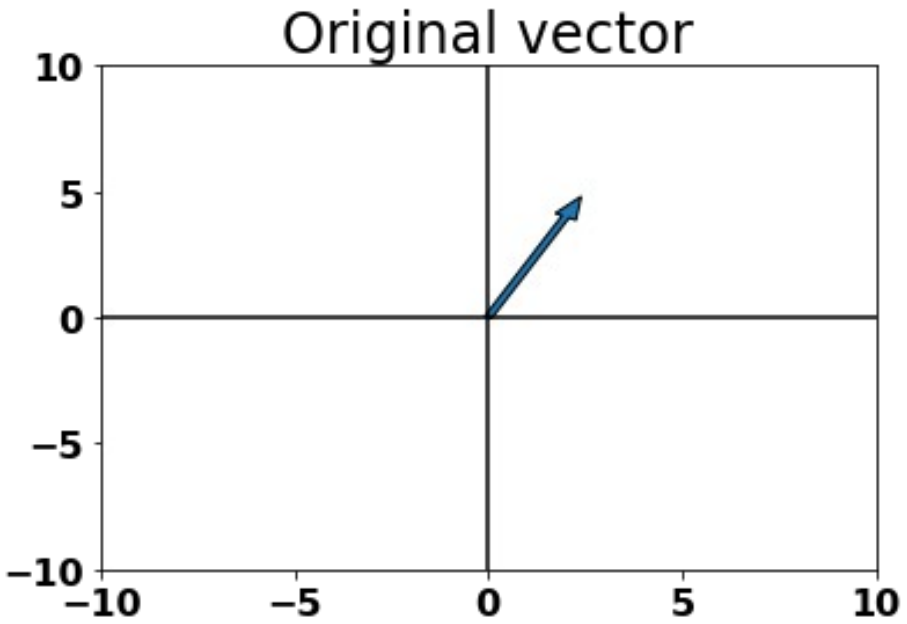

```
## plot vector (2, 4)
v = (2, 4)
plot_vector(v, 'Original vector')
```

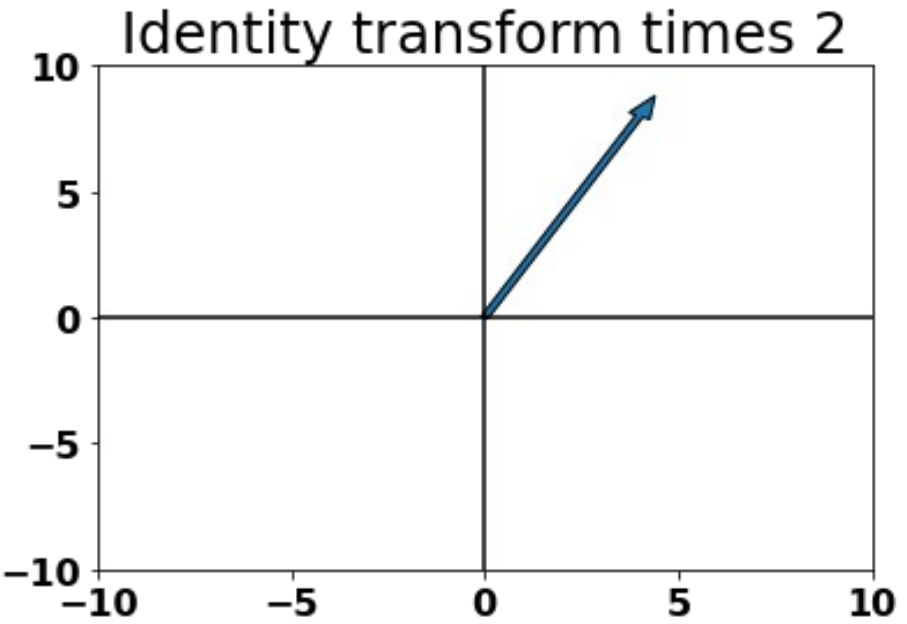



```
A = np.identity(2)
print('A = \n' + str(A))
plot_vector(A @ v, 'Identity transform')
```

```
A =
[[1. 0.]
 [0. 1.]]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

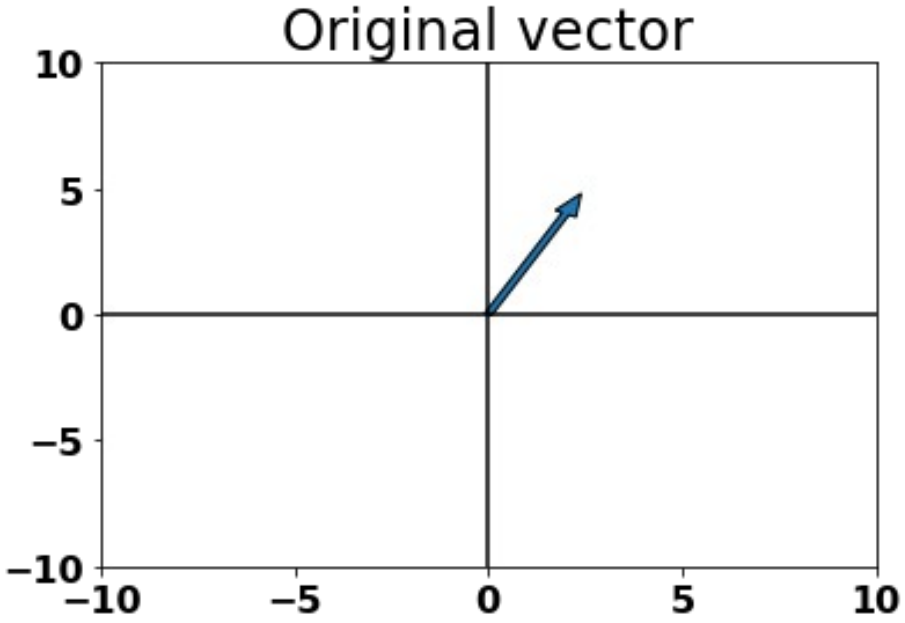


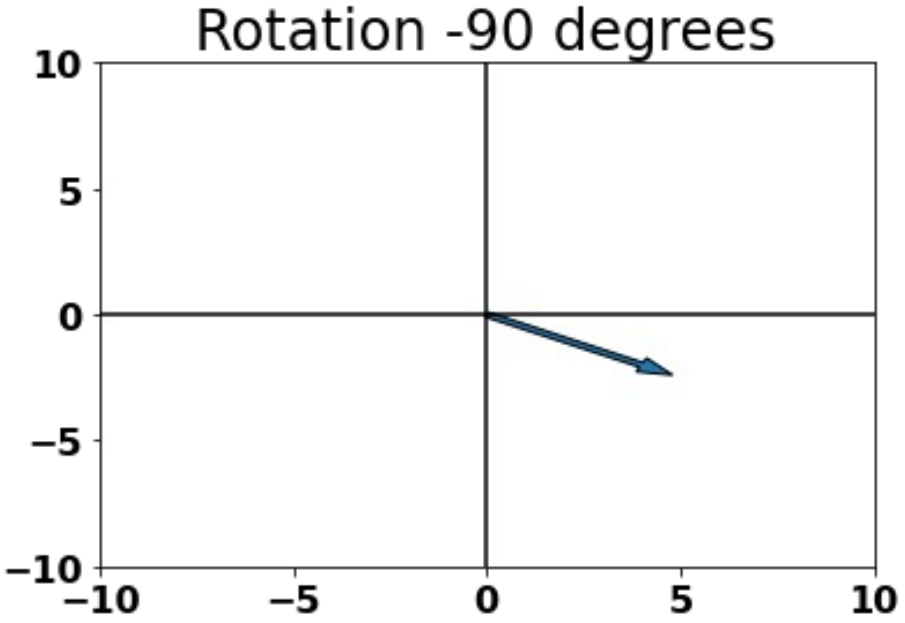



```
## transform with identity matrix times 2
A = 2 * np.identity(2)
print('A = \n' + str(A))
plot_vector(A @ v, 'Identity transform times 2')
```

```
A =
[[2. 0.]
 [0. 2.]]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

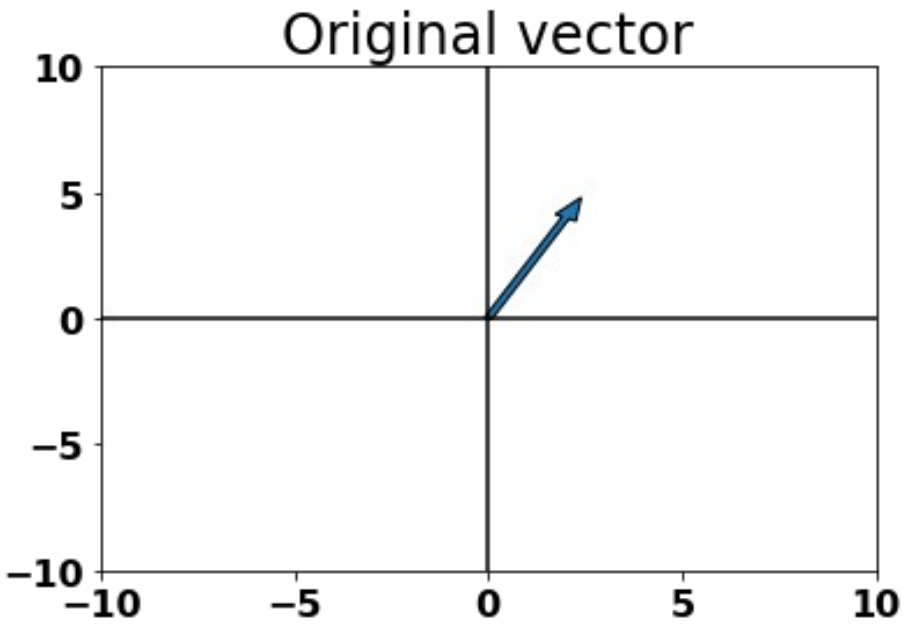


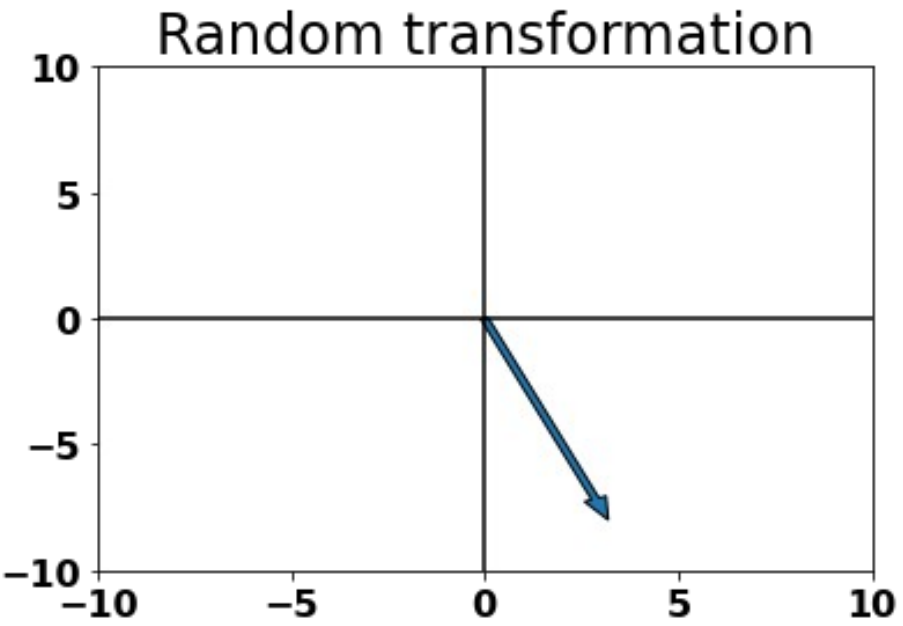



```
A = np.array([[0, 1], [-1, 0]])
print('A = \n' + str(A))
plot_vector(A @ v, 'Rotation -90 degrees')
```

```
A =
[[ 0  1]
 [-1  0]]
```

CC BY Licence 4.0: Lau Møller Andersen 2026






```
np.random.seed(1)
A = np.reshape(np.random.uniform(low=-2, high=2, size=4), newshape=(2, 2))
print('A = \n' + str(A))
plot_vector(A @ v, 'Random transformation')
```

```
A =
[[-0.33191198  0.88129797]
 [-1.9995425  -0.79066971]]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

In [142]:
```
## eigenvalues and eigenvectors
eigen_vals, eigen_vecs = np.linalg.eig(cov_mat)
print(eigen_vals.shape)
print(eigen_vecs.shape)
```

```
(13,)
(13, 13)
```

CC BY Licence 4.0: Lau Møller Andersen 2026

In [17]:
```
# checking whether the equation holds
evec_0 = eigen_vecs[:, 0]
eval_0 = eigen_vals[0]

mat_trans = cov_mat @ evec_0 ## A times x
scalar_trans = eval_0 * evec_0 # lambda times x
print(mat_trans)
print(scalar_trans)

print(np.isclose(mat_trans, scalar_trans)) ## instead of using "==", due to rounding error
```

```
[ 0.7176924  -1.18513985 -0.14644842 -1.24846827  0.5909797   1.90479358
  2.07074217 -1.49875647  1.49568723 -0.48283127  1.46928422  1.80140436
  1.43147537]
[ 0.7176924  -1.18513985 -0.14644842 -1.24846827  0.5909797   1.90479358
  2.07074217 -1.49875647  1.49568723 -0.48283127  1.46928422  1.80140436
  1.43147537]
[ True  True  True  True  True  True  True  True  True  True  True  True
  True]
```

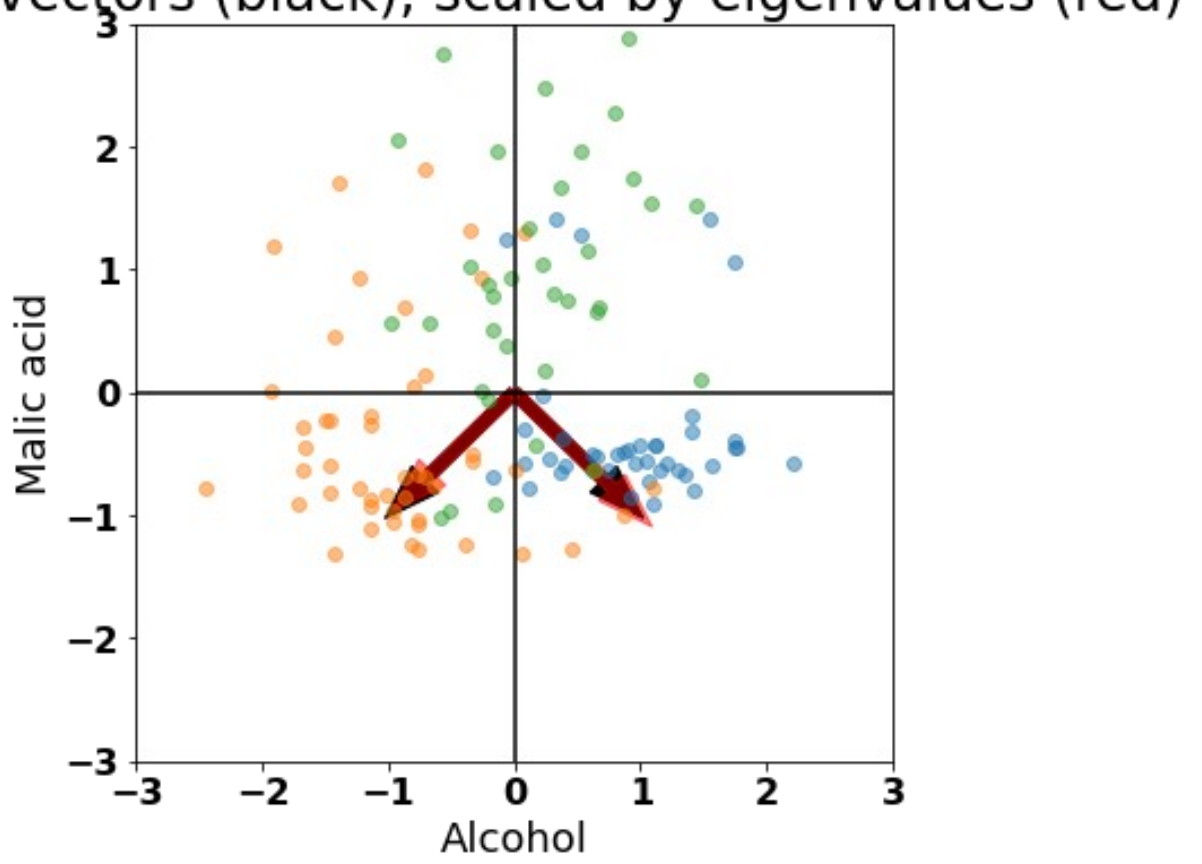
CC BY Licence 4.0: Lau Møller Andersen 2026

$$\Sigma \boldsymbol{v} = \lambda \boldsymbol{v}$$

```
Eigenvalues:
[0.92015307 1.09610709]

Eigenvectors:
 [[-0.70710678 -0.70710678]
 [ 0.70710678 -0.70710678]]

Covariance matrix:
[[1.00813008 0.08797701]
 [0.08797701 1.00813008]]
```

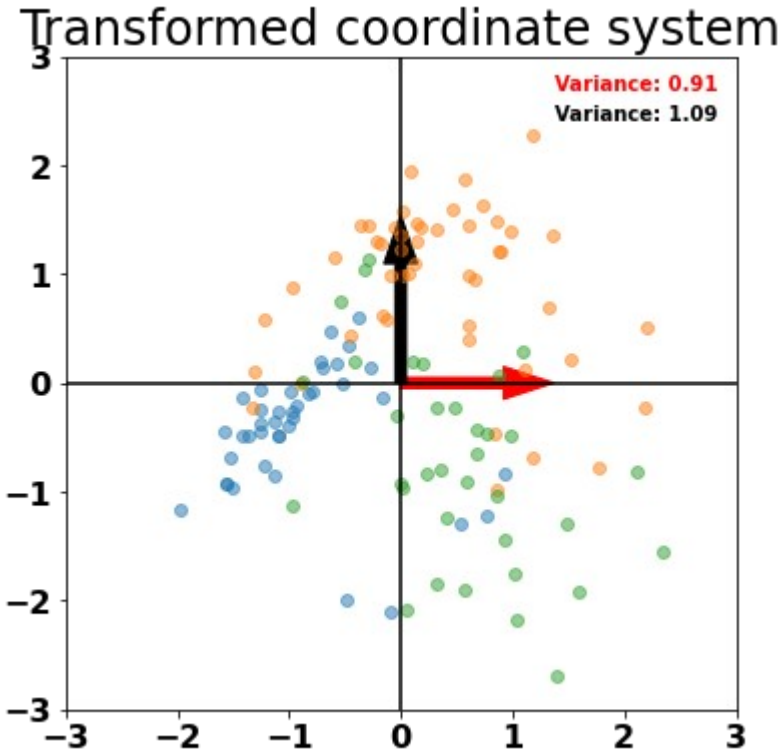


CC BY Licence 4.0: Lau Møller Andersen 2026

Applying a transformation

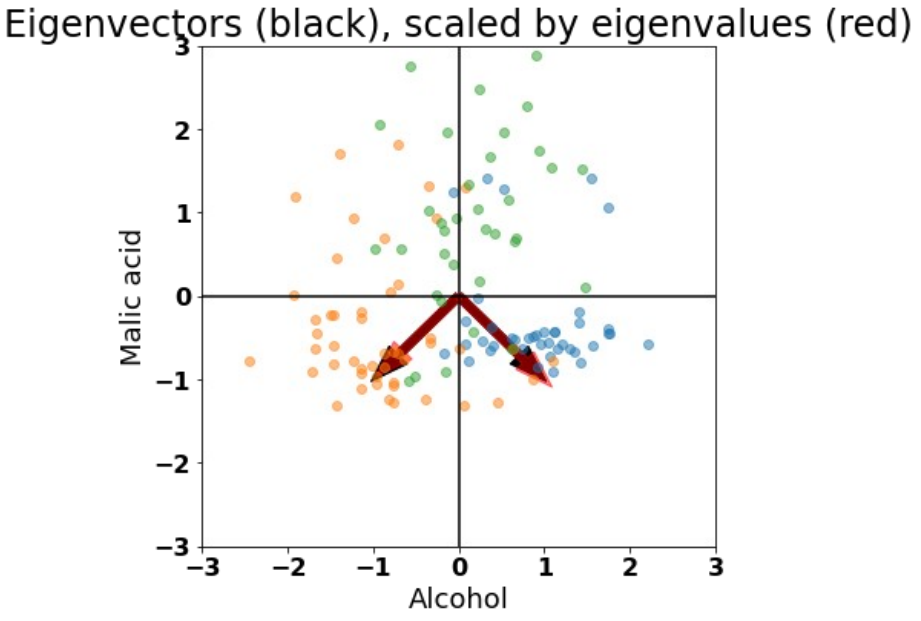
```
radian = np.arctan(reduced_eigen_vecs[1, 1] / reduced_eigen_vecs[1, 0])

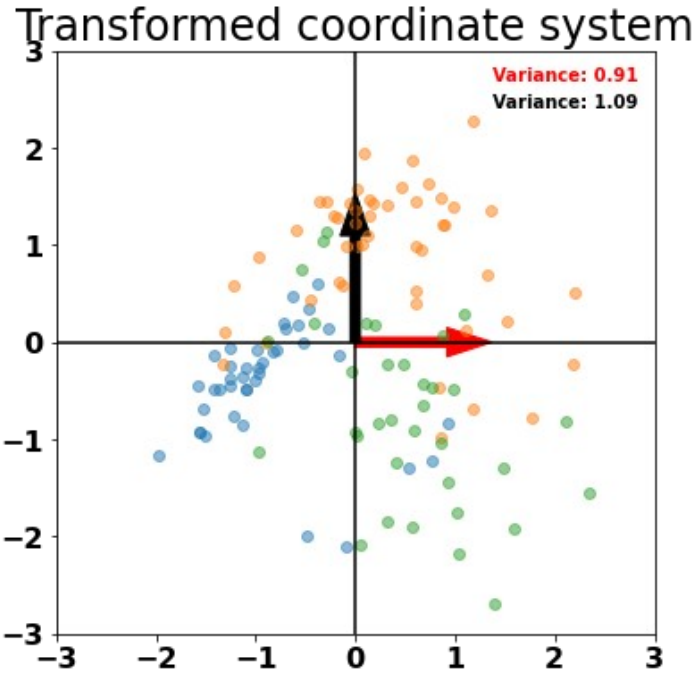

A = np.array([
                [np.cos(radian), np.sin(radian)],
                [-np.sin(radian), np.cos(radian)]
            ])
```



CC BY Licence 4.0: Lau Møller Andersen 2026

For comparison





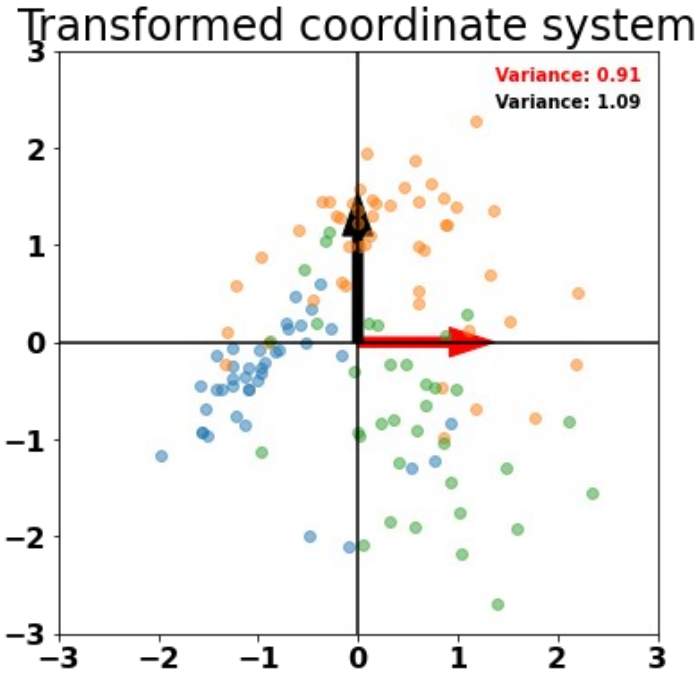
CC BY Licence 4.0: Lau Møller Andersen 2026

In the case where $k=d$...

4) Select k eigenvectors that correspond to the k largest eigenvalues where k is the dimensionality of the new feature subspace ($k \leq d$)
5) Construct a projection matrix $\boldsymbol{W}$ from the “top” k eigenvectors
6) Transform the d-dimensional input dataset $\boldsymbol{x}$ using the projection matrix $\boldsymbol{W}$ to obtain the new k-dimensional feature subspace

CC BY Licence 4.0: Lau Møller Andersen 2026

... applying steps 4, 5 & 6 results in:



note: the x-axis and the y-axis should be swapped around, as the PC explaining the most variance should be on the x-axis

CC BY Licence 4.0: Lau Møller Andersen 2026

this is **not** feature reduction, however...

CC BY Licence 4.0: Lau Møller Andersen 2026

“reducing” to a new subspace (*k*=*d*) (4)

Eigenvalues for the **full** covariance matrix

```
## going back to the full feature matrix
print(eigen_vals)
```

```
[4.8923083  2.46635032 1.42809973 1.01233462 0.84906459 0.60181514
 0.52251546 0.08414846 0.33051429 0.29595018 0.16831254 0.21432212
 0.2399553 ]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Variance explained ratio

$$\frac{\lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

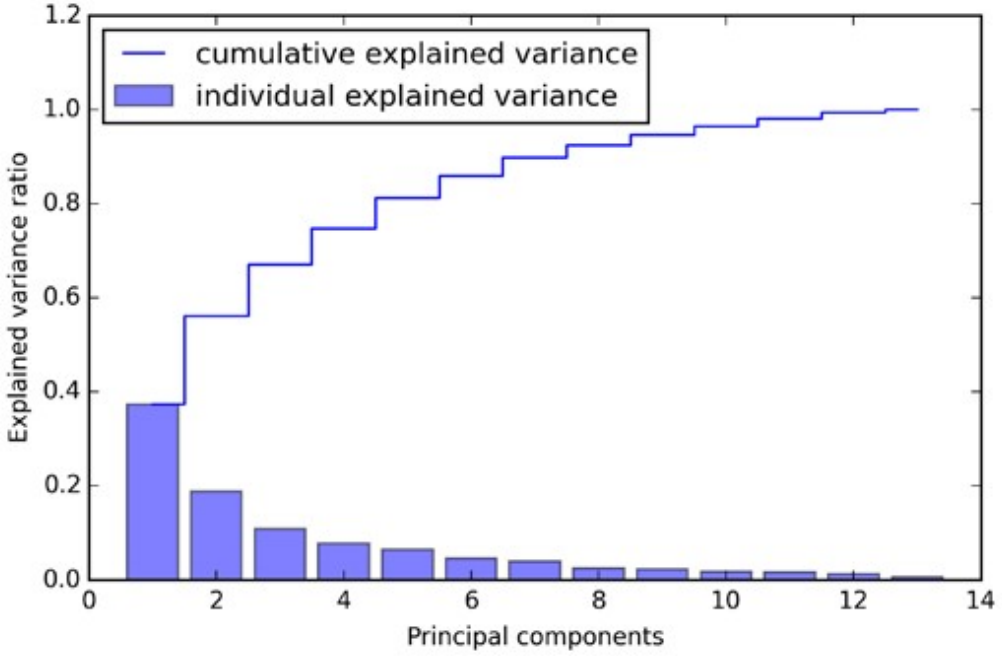
$$\frac{\lambda_1}{\sum_{j=1}^{d} \lambda_j} = 37\%$$

$$\lambda_1 = max(\lambda_j)$$
$$\lambda_7 = min(\lambda_j)$$

$$\frac{\lambda_7}{\sum_{j=1}^{d} \lambda_j} = 0.64\%$$

CC BY Licence 4.0: Lau Møller Andersen 2026

Sorted explained variance



(p. 132: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Setting $k = 2$

CC BY Licence 4.0: Lau Møller Andersen 2026

Construct a projection matrix ***W*** from the "top" *k* eigenvectors (5)

```
print('Weight matrix:\n', W)
```

```
Weight matrix:
 [[ 0.14669811  0.50417079]
 [-0.24224554  0.24216889]
 [-0.02993442  0.28698484]
 [-0.25519002 -0.06468718]
 [ 0.12079772  0.22995385]
 [ 0.38934455  0.09363991]
 [ 0.42326486  0.01088622]
 [-0.30634956  0.01870216]
 [ 0.30572219  0.03040352]
 [-0.09869191  0.54527081]
 [ 0.30032535 -0.27924322]
 [ 0.36821154 -0.174365  ]
 [ 0.29259713  0.36315461]]
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Transform the *d*-dimensional input dataset ***X*** using the projection matrix ***W*** to obtain the new *k*-dimensional feature subspace (6)

CC BY Licence 4.0: Lau Møller Andersen 2026

$$\boldsymbol{x} = [x_1, x_2, \ldots, x_d], \boldsymbol{x} \in \mathbb{R}^{\boldsymbol{d}}$$

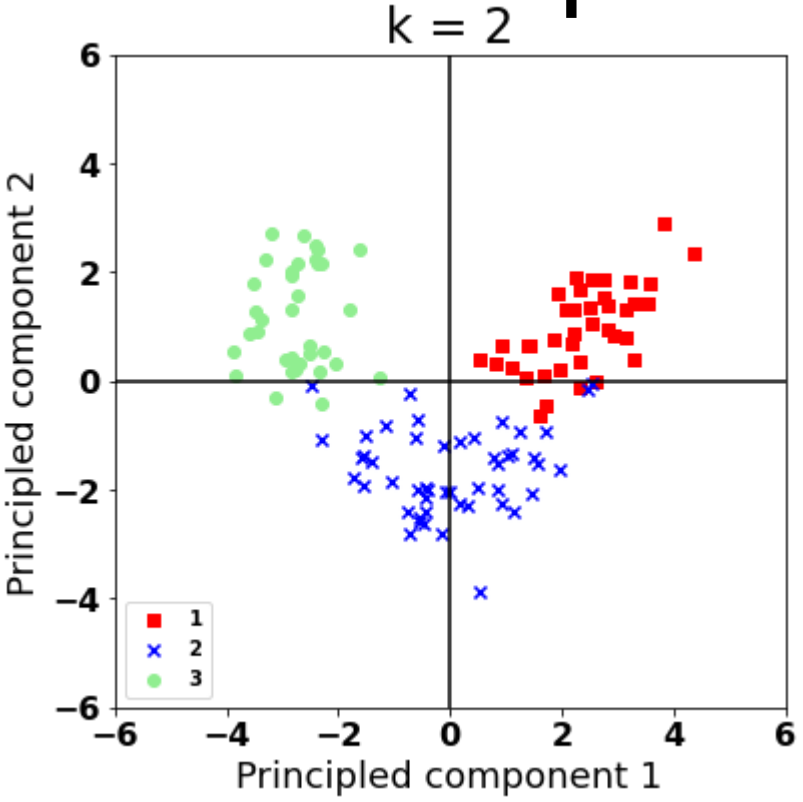
$$\downarrow \boldsymbol{xW}, \boldsymbol{W} \in \mathbb{R}^{d \times k}$$

$$\boldsymbol{z} = [z_1, z_2, \ldots, z_k], \boldsymbol{z} \in \mathbb{R}^{\boldsymbol{k}}$$

$$\boldsymbol{Z} = \boldsymbol{XW}$$

CC BY Licence 4.0: Lau Møller Andersen 2026

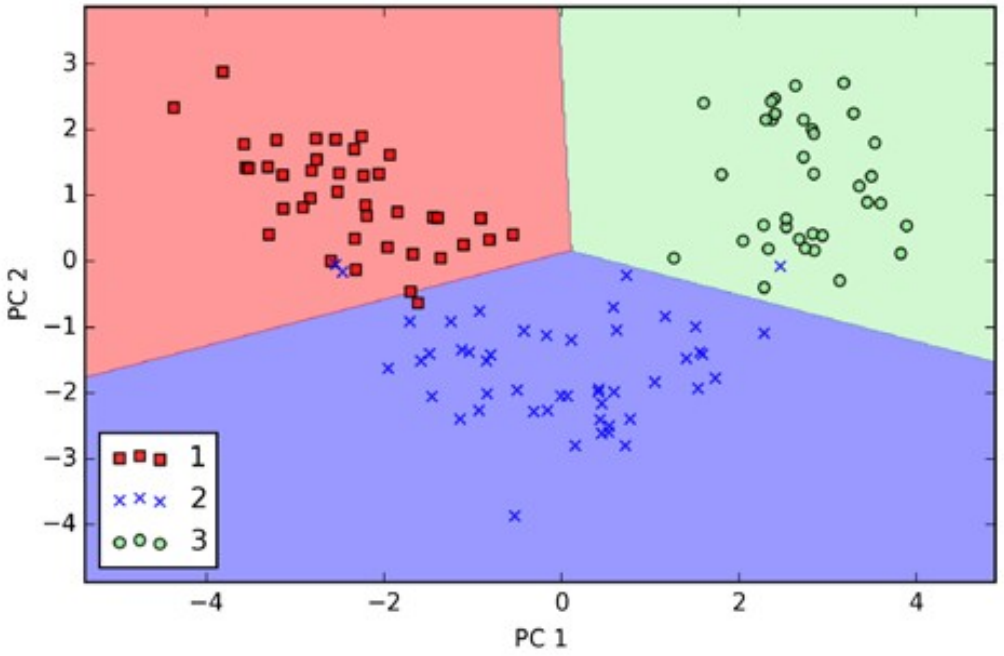
Reduced space



We can now do logistic regression in this space

CC BY Licence 4.0: Lau Møller Andersen 2026

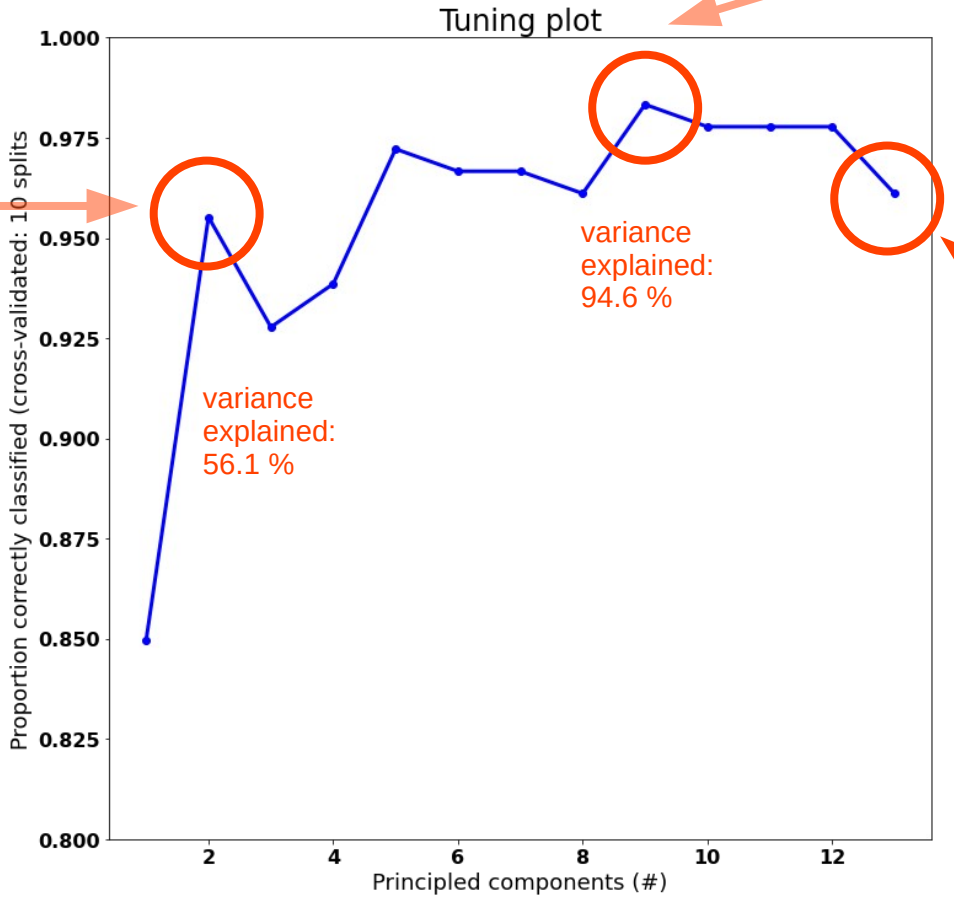
Logistic regression on reduced space (k = 2)



(p. 136: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

peak
$k = 9$



$k = 2$

What does this analysis correspond to?

CC BY Licence 4.0: Lau Møller Andersen 2026

Summary

- PCA is a general and versatile method that can be applied to datasets to reduce their complexity
- Treating *n_components* as a hyperparameter can help one do more generalisable fits.
- PCA can also be used in its own right to understand which dimensions are most important for explaining the effects seen (remember the video)

CC BY Licence 4.0: Lau Møller Andersen 2026

Learning goals and outline

Dimensionality Reduction, Principled Component Analysis (PCA)

1) Learning how we can extract the features that explain the most variance
2) Understanding how that can improve classification
3) Get acquainted with the concept of a eigenvector

CC BY Licence 4.0: Lau Møller Andersen 2026

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects?

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous?

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data?

Week 47:

Lesson 9: Neural networks

Class 9: Code review

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything!

CC BY Licence 4.0: Lau Møller Andersen 2026

Next time

- Neural networks
  - can be seen as multi-layer perceptrons, which we have been introduced to
  - can also be seen as a string of regressions, logistic or linear
  - Very powerful, but also much more complex, and thus less interpretable than our simple regressions
  - we will only be looking at networks with one hidden layer

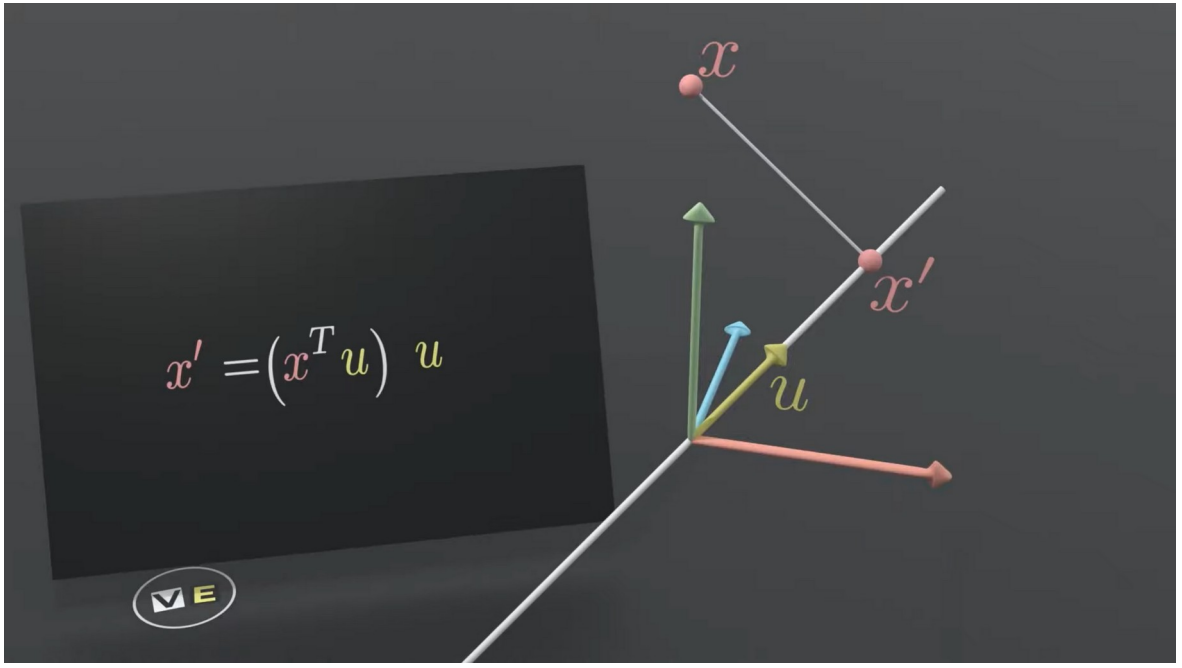
CC BY Licence 4.0: Lau Møller Andersen 2026

Reading questions

- What is forward propagation?
- What is backpropagation?
- What are the different dimensions: *m*, *n*, *h*, *t* etc.
- What do each the different *methods* (def) of the class *NeuralNetMLP* do?
- How does gradient checking work?

CC BY Licence 4.0: Lau Møller Andersen 2026

Video again?



/home/lau/Videoklip/methods3/Principal Component Analysis (PCA) [FD4DeN81ODY].mp4
Principal Component Analysis by Visually Explained (Youtube)

CC BY Licence 4.0: Lau Møller Andersen 2026